Claude skills: choose a workflow, author it clearly and test its value

From collection to a working skill

The directory reel lists community skills, the rules reel discusses skill structure, and the engineering reel adds evaluation and prompt pruning. Their shared task is building reliable reusable instructions. Installing a large collection is not the same as demonstrating that a skill improves a particular workflow.

The official authoring guide (https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices) recommends concise main instructions, direct references and trigger descriptions that explain when the skill applies. Treat length and structure guidance as authoring practices, not evidence that every longer file fails. Match procedural rigidity to the task's fragility; subjective review and a database migration need different degrees of freedom.

Choose from the original collection

The source's LinkedIn short links were broken during the original check. The replacement repositories are name-based matches with confidence notes in the individual guide, not recovered redirects. Useful families include Superpowers (https://github.com/obra/superpowers) for an engineering process, Matt Pocock's skills (https://github.com/mattpocock/skills) for developer workflows, UI/UX Pro Max (https://github.com/nextlevelbuilder/ui-ux-pro-max-skill) for design reference, and Web Quality (https://github.com/addyosmani/web-quality-skills) for audits. Other named resources include Humanizer (https://github.com/blader/humanizer), Caveman (https://github.com/JuliusBrussee/caveman), Excalidraw (https://github.com/coleam00/excalidraw-diagram-skill), Remotion (https://github.com/remotion-dev/skills) and Vercel skills (https://github.com/vercel-labs/skills).

Author and test one workflow

Define a concrete trigger, required inputs, useful output and a stopping condition. Put shared operating rules in the main skill; link longer task references directly. State dependencies and error handling for scripts. Inspect installation instructions in the selected repository instead of assuming one marketplace command fits every provider.

Run positive and negative trigger examples, missing-input cases and a realistic task with acceptance criteria. Record false activations and omitted steps. Then remove one instruction at a time and rerun the same cases. Keep instructions that demonstrably change necessary behavior; preserve important constraints even when a tiny test set fails to reveal their value.

Avoid overreading the engineering reel

Claims about an internal 80% prompt reduction are not a universal instruction to delete 80% of your own context at every model release. Undocumented environment switches and internal harness options in the source account are not a supported public interface. The important transferable practice is regression testing after configuration changes.

Proposed project and prompt

Build a local skill audit with a manifest, trigger examples, reference checks and a before/after evaluation report. Check broken paths, vague descriptions and scripts with unstated prerequisites. Prefer one useful skill over mass installation.

```
Audit one developer skill against its intended tasks. Create positive and
negative trigger examples, verify direct reference paths, document script
dependencies, and compare baseline vs revised instructions on the same
acceptance criteria. Propose changes with evidence and retain a rollback.
```

The original twelve-entry directory, command examples and source-specific prompts remain in Files and Sources; the unified guide supplies the end-to-end selection, authoring and evaluation process.

Original sources

- https://www.instagram.com/p/DdcDA1sN8eS/
- https://www.instagram.com/reel/DeQYD7-CXLk/
- https://www.instagram.com/reel/DeCZn6JMmYf/